

slam_toolbox Localization and Navigation

slam_toolbox Localization and Navigation

1. Course Content
2. Principle Introduction
 - 2.1 Introduction
 - 2.2 Difference Between the Two Localization Methods
3. Start the Agent
4. Run the Example
 - 4.1 Preparations
 - 4.2 Start the Navigation Node
5. Rapid Relocalization
 - 5.1 Odometry Cumulative Error
 - 5.2 Robot Kidnapping
6. Source Code Analysis
 - 6.1 View TF Tree
 - 6.2 Launch File

1. Course Content

1. Run the sample program to perform navigation using slam_toolbox in localization mode and quickly recover the robot's precise position after a kidnapping event.

2. Principle Introduction

2.1 Introduction

- The default localization algorithm in Navigation 2 is AMCL (Adaptive Monte Carlo Localization). This involves replacing the default AMCL localization method with the pure localization mode of slam_toolbox, while other navigation functions remain unchanged.

2.2 Difference Between the Two Localization Methods

- **AMCL:** Suitable for scenarios with a known map, such as when a robot performs repetitive tasks or navigation in an indoor environment with a pre-built map. AMCL can be used to determine its own position.
- **slam_toolbox:** Compared to AMCL's particle filter localization, which must estimate position through probability distribution while moving, slam_toolbox's pure localization can use a pose graph for rapid scan-to-map matching to solve for a precise global position.

3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1765183764.822476] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1765183764.826313] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x006(2), participant_
id: 0x000(1)
[1765183764.829482] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1765183764.834154] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1765183764.838233] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x007(2), participant_
id: 0x000(1)
[1765183764.842504] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1765183764.847468] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

4. Run the Example

4.1 Preparations

- A grid map and a pose graph map in `posegraph` format are required. To build a pose graph map, refer to the `09-LifeLong Mapping` chapter.

4.2 Start the Navigation Node

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

(ORIN board)

```
ros2 launch m3_bringup nav2.launch.py map:=map.yaml pose_map:=map
localization_mode:=slam_toolbox
```

(RDK & PI5 boards) Run in separate steps, with visualization launched on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`,

On the virtual machine

```
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz
```

On the host machine

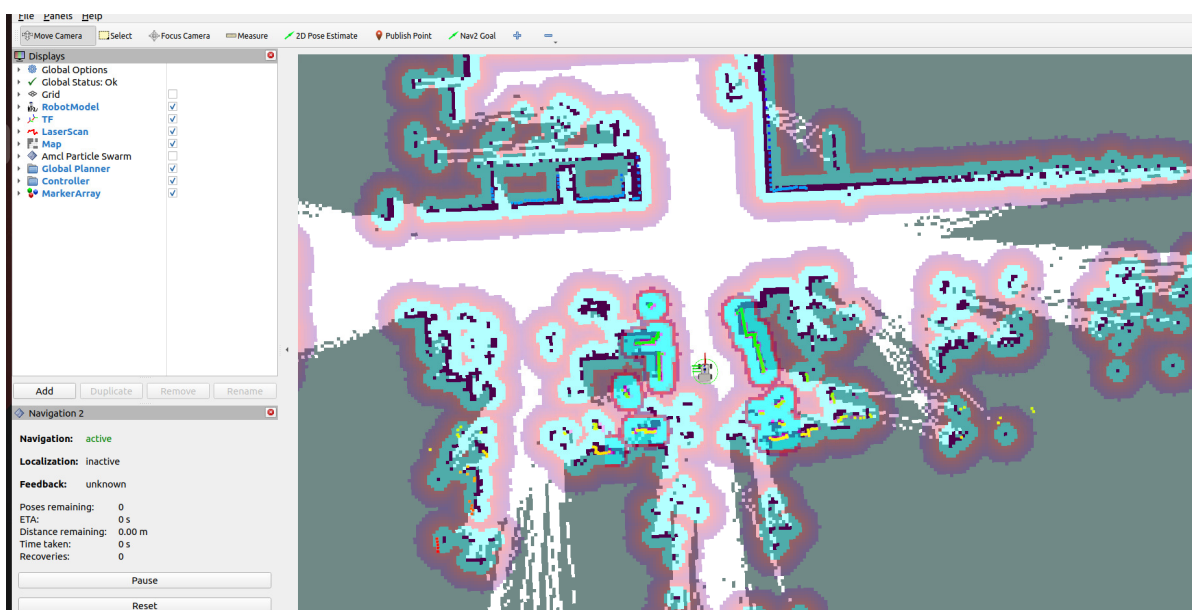
```
ros2 launch m3_bringup nav2.launch.py map:=map.yaml pose_map:=map  
localization_mode:=slam_toolbox gui:=False
```

[!TIP]

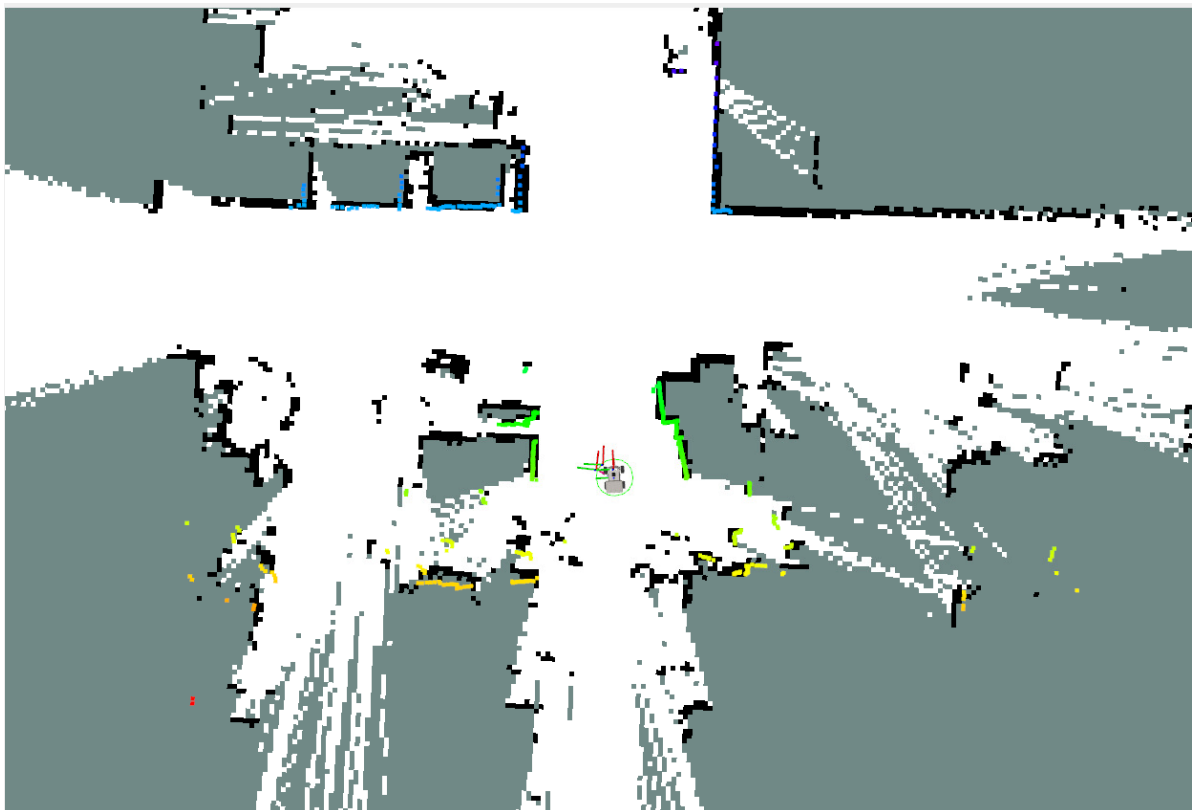
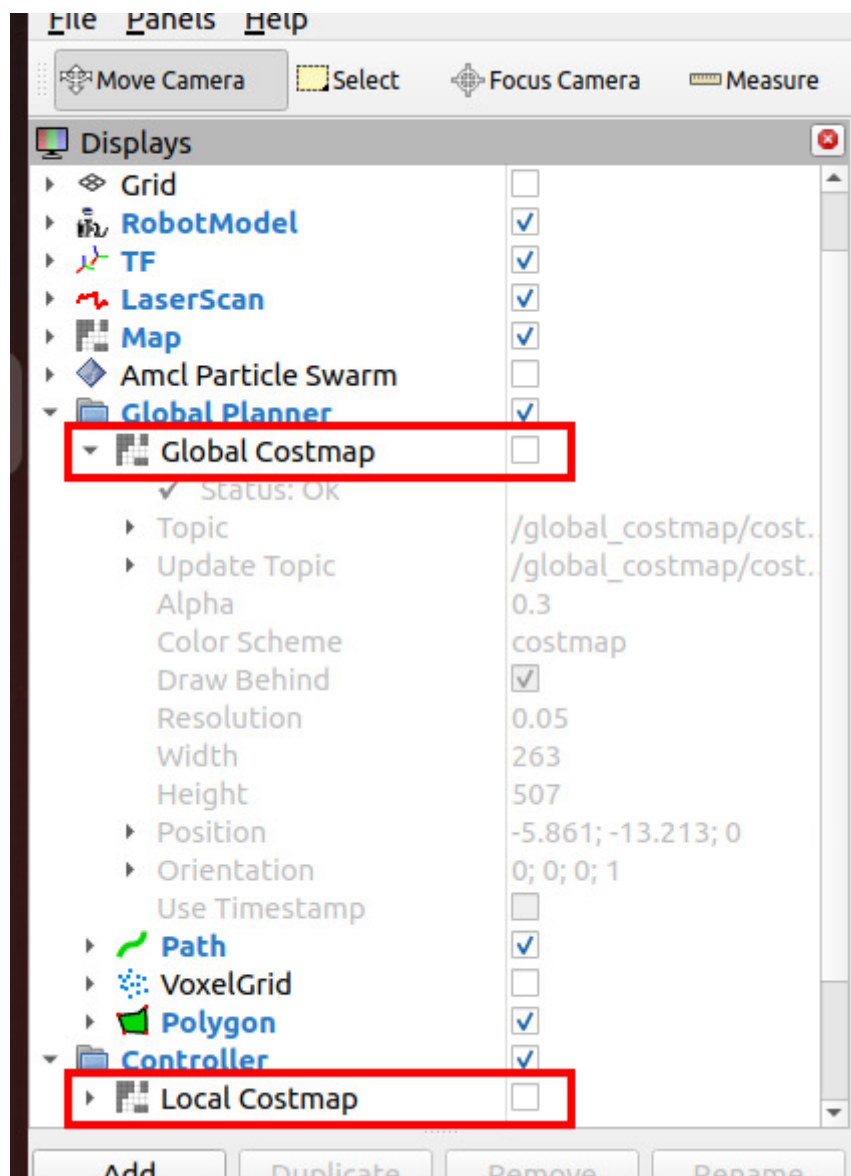
Launch Parameter Descriptions

- `localization_mode`
 - The global localization method. Here, `slam_toolbox`'s localization method is selected, which can achieve higher-precision global localization using a pose graph.
- `pose_map`
 - The name of the pose graph map built by `slam_toolbox`. If `slam_toolbox` is chosen as the localization method, this must be provided. The default is `map`.
- `map`
 - The name of the grid map, default is `map.yaml`.

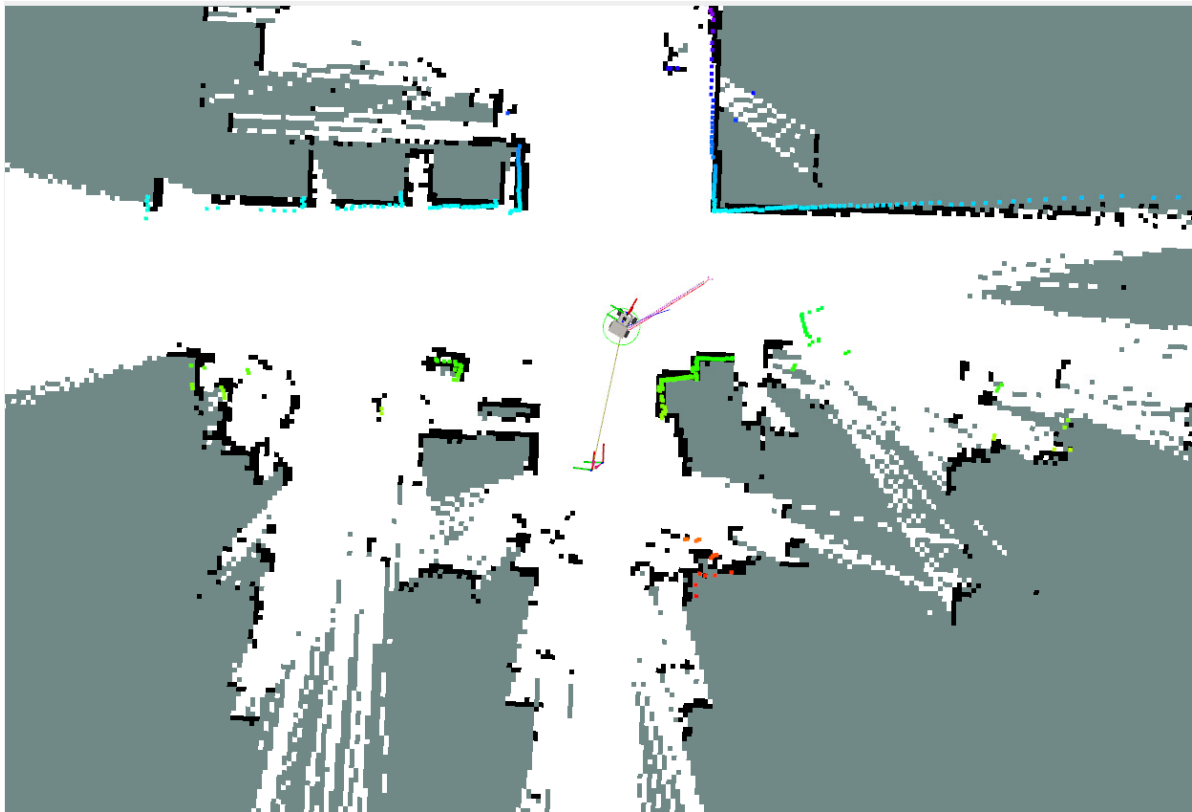
- Near the map's origin, localization can be completed without providing an initial pose.



- In the left toolbar, you can uncheck the global and local costmaps to get a clearer view of the laser point cloud and the environment's contours.



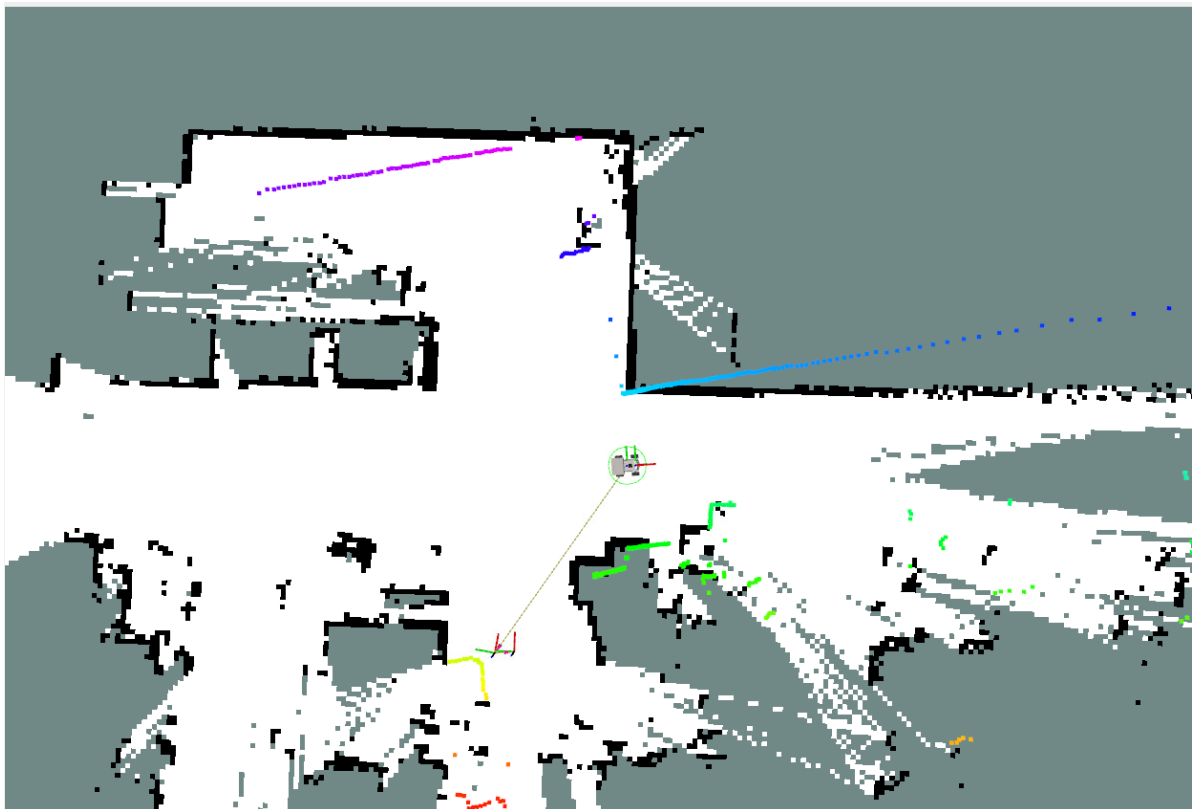
- Navigation is still performed by specifying a target pose with `Nav2 Goal`.



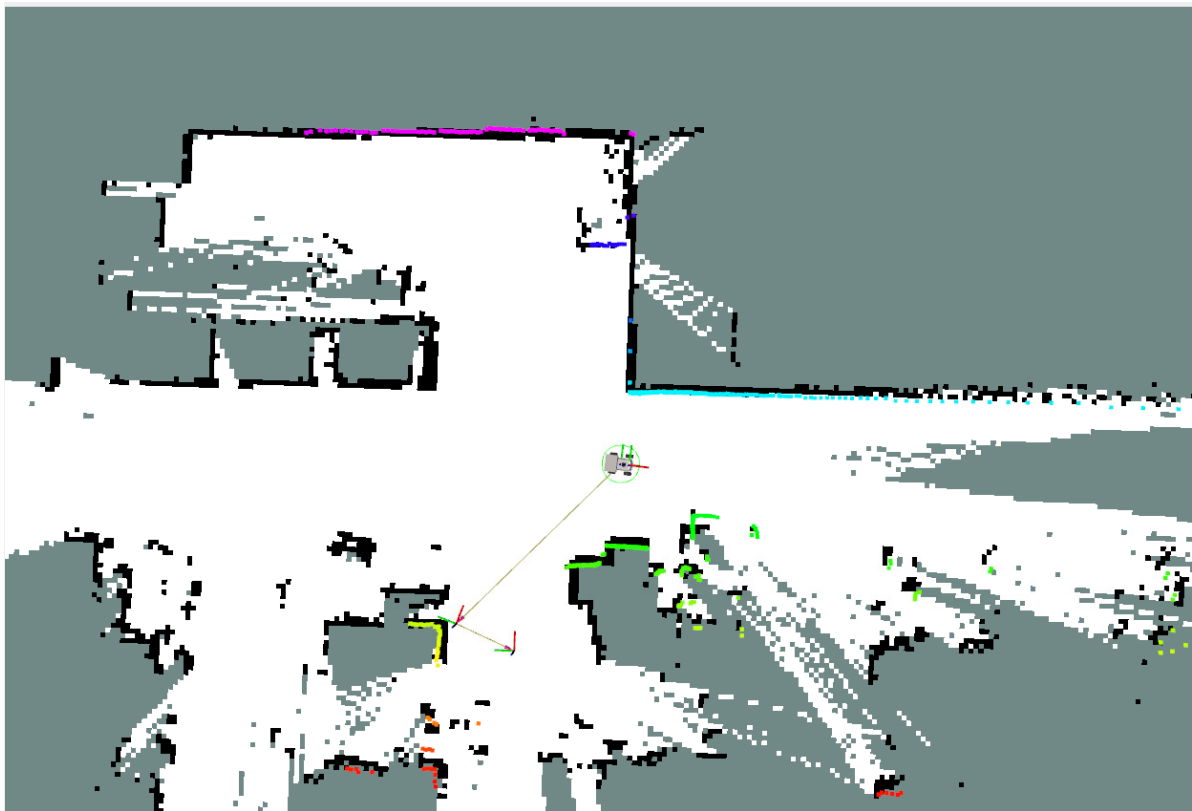
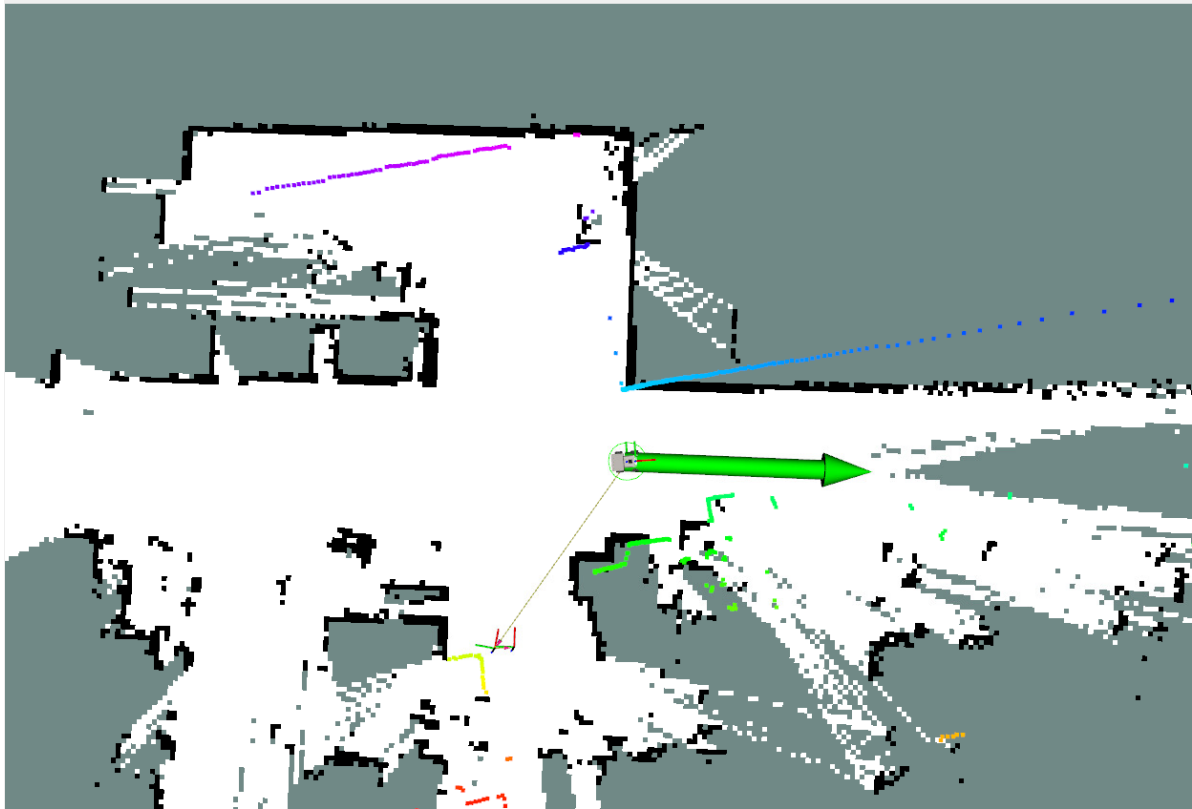
5. Rapid Relocalization

5.1 Odometry Cumulative Error

- Odometry can accumulate errors or experience errors in local positioning due to wheel slippage. At this point, the laser point cloud and the environment's contours will no longer match, as shown in the figure below:



- With a pose graph, you only need to provide an approximate pose using the `2D Pose Estimate` tool to achieve precise localization.



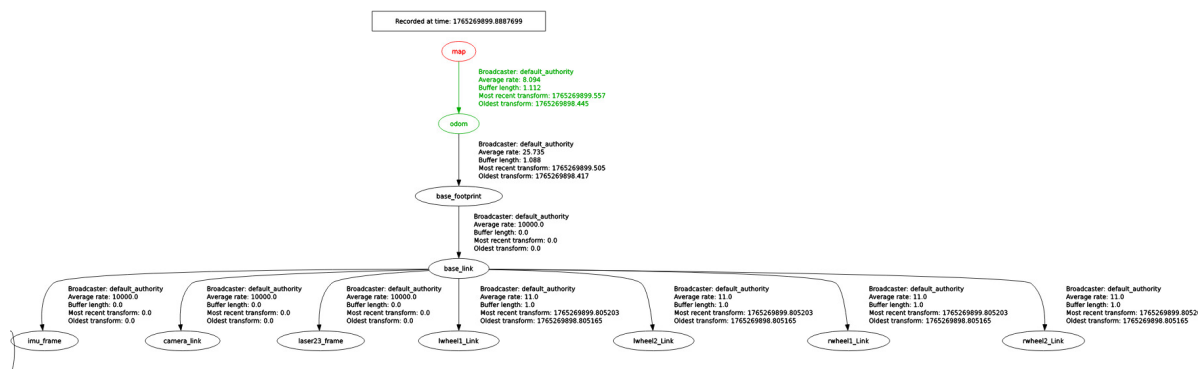
6. Source Code Analysis

6.1 View TF Tree

In the terminal, enter:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

- Here, the TF transformation from the `map` coordinate frame to the `odom` coordinate frame is provided by `slam_toolbox`.



6.2 Launch File

- Launch file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/navigation/nav2.launch.py
```

- The key code snippet below starts the Navigation2 stack node and the `slam_toolbox` pure localization node.

```
# slamtoolbox positioning navigation
slamtoolbox_localization_nav = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'slam_toolbox' and '", slam, "' == 'False' "]])),
    actions=[
        PushRosNamespace(namespace=namespace),
        # Start navigation2 navigation file (no global positioning)
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(nav2_launch_mod),
            launch_arguments={
                "map": map_dir,
                "params_file": params_file,
                "use_sim_time": use_sim_time,
            }.items()),
```



```

        # Start slamtoolbox positioning node
        IncludeLaunchDescription(

PythonLaunchDescriptionSource(slamtoolbox_localization_launch_file),
        launch_arguments={
            "map": pose_map,
            "use_sim_time": use_sim_time,
        }.items(),
    ),
    # Log Printing
    LogInfo(msg=[Fore.GREEN+'=====Robot-M3 Navigation
start with slamToolbox localization====='],Fore.RESET])
]
)

```

- Parameter file path. If you need to configure the slam_toolbox parameters for pure localization mode, the path is as follows:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/mapper_params_localization.yaml
```

```

slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ROS Parameters
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    mode: localization #localization

    # if you'd like to start localizing on bringup in a map and pose
    map_file_name: ""
    #map_start_pose: [5.0, 1.0, 0.0]
    map_start_at_dock: true

    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.02 #if 0 never publishes odometry
    map_update_interval: 5.0
    resolution: 0.05
    min_laser_range: 0.05 #for rastering images
    max_laser_range: 12.0 #for rastering images
    minimum_time_interval: 0.5
    transform_timeout: 0.2
    tf_buffer_duration: 30.

```

```
stack_size_to_use: 40000000 #// program needs a larger stack size to  
serialize large maps
```

```
# General Parameters
```

```
use_scan_matching: true  
use_scan_barycenter: true  
minimum_travel_distance: 0.5  
minimum_travel_heading: 0.5  
scan_buffer_size: 3  
scan_buffer_maximum_scan_distance: 10.0  
link_match_minimum_response_fine: 0.1  
link_scan_maximum_distance: 1.5  
do_loop_closing: true  
loop_match_minimum_chain_size: 3  
loop_match_maximum_variance_coarse: 3.0  
loop_match_minimum_response_coarse: 0.35  
loop_match_minimum_response_fine: 0.45
```

```
# Correlation Parameters - Correlation Parameters
```

```
correlation_search_space_dimension: 0.5  
correlation_search_space_resolution: 0.01  
correlation_search_space_smear_deviation: 0.1
```

```
# Correlation Parameters - Loop Closure Parameters
```

```
loop_search_space_dimension: 8.0  
loop_search_space_resolution: 0.05  
loop_search_space_smear_deviation: 0.03  
loop_search_maximum_distance: 3.0
```

```
# Scan Matcher Parameters
```

```
distance_variance_penalty: 0.5  
angle_variance_penalty: 1.0  
  
fine_search_angle_offset: 0.00349  
coarse_search_angle_offset: 0.349  
coarse_angle_resolution: 0.0349  
minimum_angle_penalty: 0.9  
minimum_distance_penalty: 0.5  
use_response_expansion: true  
min_pass_through: 2  
occupancy_threshold: 0.1
```

